



北京師範大學
BEIJING NORMAL UNIVERSITY

计算方法 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 7 月 1 日

目录

1	环境说明	3
2	问题描述	3
3	根据 n 直接计算	3
3.1	复化梯形公式	3
3.2	复化 Simpson 公式	3
3.3	复化 Cotes 公式 (五点 Cotes)	4
4	自适应复化积分函数	4
5	估计收敛阶数	4
5.1	定义	4
5.2	程序所用方法推导	4
6	结果展示	5
6.1	函数选取	5
6.2	结果表格	5
6.3	分析	6

1 环境说明

本文全部代码为 c++ 代码，硬件为 Macbook Air M2，软件为 Xcode。在 Xcode 内部进行基于命令行的编译运行。

本实验报告使用 L^AT_EX 进行制作，基于 overleaf 网站进行在线写作。

2 问题描述

编写一个计算一维的数值积分的函数，要求把被积函数 f 、积分区间 $[a, b]$ 、小区间个数 n ，精度要求 ϵ 以及采用的数值积分方法（复化梯形、复化 Simpson、复化 Cotes）作为参数。

利用上述函数找 2,3 个算例，分别验证各种积分公式的收敛阶。

3 根据 n 直接计算

数值积分是将连续函数的积分近似为一组离散点函数值加权和。我们实现了三种经典的复化积分方法：复化梯形公式、复化 Simpson 公式与复化 Cotes 公式。

3.1 复化梯形公式

复化梯形公式将区间 $[a, b]$ 划分为 n 个等长小区间，每个区间用一条直线近似函数图像，其积分公式为：

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$

其中 $h = \frac{b-a}{n}$ 。

3.2 复化 Simpson 公式

Simpson 公式基于抛物线拟合每两个小区间，要求 n 为偶数，其公式为：

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + 4 \sum_{\text{odd } i} f(x_i) + 2 \sum_{\text{even } i} f(x_i) + f(b) \right]$$

其中 $h = \frac{b-a}{n}$ 。

这里的 n 与上课 PPT 的不一致，是为了方便使用整数下标不含分数。与上课的 n_1 的关系为： $n = 2 * n_1$ 。

3.3 复化 Cotes 公式 (五点 Cotes)

Cotes 公式使用四次插值多项式 (即五个节点), 每四个小区间构成一组。公式为:

$$\int_{x_0}^{x_4} f(x) dx \approx \frac{2h}{45} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)]$$

这里的 n 与上课 PPT 的不一致, 是为了方便使用整数下标不含分数。与上课的 n_2 的关系为: $n = 4 * n_2$ 。

4 自适应复化积分函数

为保证计算结果的精度, 通常会给定一个误差容限 ε , 我们实现了一个自适应积分函数 `adaptive_integrate`, 在初始子区间个数 n 基础上, 不断对 n 加倍, 直到两次积分结果误差满足:

$$|I_{2n} - I_n| < \varepsilon$$

由于对于给定的 f 和 a, b , 复化积分的误差是与 n 高度相关的, 所以把 n 和 ε 作为两个变量是不合适的。因此我们将 “ n 确定” 和 “由 ε 控制 n ” 分别封装为两个函数:

- `integrate_fixed(...)`: 输入固定 n , 计算积分值。
- `adaptive_integrate(...)`: 输入误差 ε , 从给的初始 n 开始, 寻找满足误差要求的 n 。

5 估计收敛阶数

5.1 定义

一个复化积分公式的误差为:

$$I - I_n \approx Ch^p$$

其中 $h = \frac{b-a}{n}$, p 为收敛阶, C 为某个常数。

5.2 程序所用方法推导

我们使用三个不同 n 值对应的积分结果 $I_1 = I_n, I_2 = I_{2n}, I_3 = I_{4n}$, 构造:

$$I_1 - I_2 \approx Ch^p(1 - 2^{-p})$$

$$I_2 - I_3 \approx Ch^p(2^{-p} - 4^{-p})$$

两者比值为:

$$\frac{I_1 - I_2}{I_2 - I_3} \approx \frac{1 - 2^{-p}}{2^{-p} - 4^{-p}}$$

最终对数变换得估计公式:

$$p \approx \log_2 \left(\frac{I_1 - I_2}{I_2 - I_3} \right)$$

这便是我们在程序中所使用的计算收敛阶的公式。

6 结果展示

6.1 函数选取

选取以下函数进行测试:

- $\sin(x)$
- e^x
- $\frac{\sin(x)}{x}$

积分区间设为 $[1, 2]$, 误差限为 $\varepsilon = 10^{-7}$ 。

6.2 结果表格

表 1: 各方法在不同函数上的估计阶数与自适应积分结果

函数	方法	n	积分值	阶 p	自适应积分值 ($\varepsilon = 10^{-7}$)
$\sin(x)$	Trapezoidal	4/8/16	0.951/0.955/0.956	2.00	0.9564491234 ($n = 2048$)
	Simpson	4/8/16	0.95647/0.95645/0.95644	4.01	0.9564491475 ($n = 32$)
	Cotes	4/8/16	0.95644/0.95644/0.95644	6.04	0.9564491423 ($n = 16$)
e^x	Trapezoidal	4/8/16	4.695/4.676/4.672	2.00	4.6707742937 ($n = 4096$)
	Simpson	4/8/16	4.67087/4.67078/4.67077	3.99	4.6707742720 ($n = 64$)
	Cotes	4/8/16	4.67077/4.67077/4.67077	5.96	4.6707742711 ($n = 16$)
$\frac{\sin(x)}{x}$	Trapezoidal	4/8/16	0.6586/0.6591/0.6592	2.00	0.6593298958 ($n = 1024$)
	Simpson	4/8/16	0.65933/0.65933/0.65933	4.00	0.6593299115 ($n = 16$)
	Cotes	4/8/16	0.65933/0.65933/0.65933	6.02	0.6593299062 ($n = 8$)

6.3 分析

- 所有方法的收敛阶与理论一致：梯形法 2 阶，Simpson 法 4 阶，Cotes 法 6 阶。
- Cotes 法虽然收敛最快，但每步计算开销大，适合要求极高精度时使用。
- 自适应积分中，所需 n 随着方法阶数提高迅速减少，显示高阶公式效率高。

The screenshot shows a software interface with a sidebar on the left containing a tree view of project files under '数值积分'. The main area displays the output of a program. The output is organized into sections for two functions: $\sin(x)$ and e^x , each evaluated over the interval $[1, 2]$. For each function, three methods are compared: Trapezoidal, Simpson, and Cotes. For each method, results are shown for $n=4, 8, 16$, including the integral value and the estimated order of convergence. Additionally, adaptive integration results are provided for each function, showing the result and the number of points n required. The program ends with an exit code of 0.

```
Function: sin(x), Interval: [1, 2]
Method: Trapezoidal
n=4, Integral=0.9514624397
n=8, Integral=0.952834415
n=16, Integral=0.9561377788
Estimated order of convergence: 2.0014104689
Adaptive integration (epsilon=0.0000001000):result=0.9564491234, n=2848

Method: Simpson
n=4, Integral=0.9564789541
n=8, Integral=0.9564584421
n=16, Integral=0.9564492235
Estimated order of convergence: 4.0084823072
Adaptive integration (epsilon=0.0000001000):result=0.9564491475, n=32

Method: Cotes
n=4, Integral=0.9564486316
n=8, Integral=0.9564491346
n=16, Integral=0.9564491423
Estimated order of convergence: 6.8363973744
Adaptive integration (epsilon=0.0000001000):result=0.9564491423, n=16

Function: e^x, Interval: [1, 2]
Method: Trapezoidal
n=4, Integral=4.6950759169
n=8, Integral=4.678544246
n=16, Integral=4.6722946058
Estimated order of convergence: 1.9985927214
Adaptive integration (epsilon=0.0000001000):result=4.6787742937, n=4896

Method: Simpson
n=4, Integral=4.6708748835
n=8, Integral=4.6707805938
n=16, Integral=4.6707746662
Estimated order of convergence: 3.9915754803
Adaptive integration (epsilon=0.0000001000):result=4.6707742720, n=64

Method: Cotes
n=4, Integral=4.6707742720
n=8, Integral=4.6707742720
n=16, Integral=4.6707742720
Estimated order of convergence: 6.8242922159
Adaptive integration (epsilon=0.0000001000):result=4.6707742720, n=8

Program ended with exit code: 0
```

图 1: 输出结果